

Recorrido del código

`highOrderElectroActivationFoamImplicitPETScDistributed`

Generado el 2026-08-14 a partir de

`highOrderElectroActivationFoamImplicitPETScDistributed.C` (8145 líneas)

Cómo leer esto. Los listados NO son copias: salen de `\lstinputlisting` apuntando al fuente real, con rangos que este reporte recalcula al regenerarse. El PDF de hoy muestra el código de hoy. Los números de línea del margen son los del archivo.

Índice

1. Qué resuelve este solver	2
2. El modelo iónico TNNP	2
3. Los integradores de EDO, y el único paralelismo por hilos	4
4. El residuo de estados: una colectiva que colgaba	5
5. Bloques de construcción del ensamblado	7
6. Las dos matrices de rigidez, y el defecto de la conductividad	8
7. Modo híbrido: EDO donde el frente lo pide	9
8. Tiempos de activación y muestreo del benchmark	11
9. <code>main()</code> : montaje y lazo temporal	12
10. Índice de funciones	14

1. Qué resuelve este solver

Resuelve la propagación del potencial de acción cardíaco: la ecuación de monodominio acoplada al modelo iónico **ten Tusscher–Noble–Noble–Panfilov (TNNP)**, con el objetivo de reproducir el benchmark de Niederer et al. (2012).

A diferencia del solver MMS, acá **no hay solución exacta**. La verificación es por convergencia, por invariancia frente a la descomposición, y contra referencia publicada.

En cada paso de tiempo hay dos trabajos muy distintos:

1. integrar, en cada celda o punto de Gauss, un sistema **rígido** de 19 EDO — el modelo TNNP — para obtener la corriente iónica I_{ion} ;
2. resolver el sistema lineal implícito de la difusión.

El primero domina el tiempo: la resolución lineal completa es apenas el 17–44 % del lazo. Eso explica varias decisiones del archivo, entre ellas que la única región OpenMP esté sobre las EDO y no sobre el ensamblado.

Dos ejes de paralelismo, en lugares distintos del archivo: **MPI** reparte la malla entre ranks y toca todas las fases; **OpenMP** reparte el lazo de puntos de integración dentro de un rank y sólo toca las EDO.

2. El modelo iónico TNNP

El modelo vive en una clase dentro del archivo, con sus 19 estados y sus variables algebraicas.

`computeRates` evalúa las derivadas; `computeVariables` las cantidades algebraicas; `calculateCurrent` devuelve I_{ion} .

`protectState` es la protección numérica: acota compuertas y concentraciones a rangos físicos. Lleva un contador de correcciones, y ése es el único estado compartido de todo el camino de las EDO — por eso su incremento va bajo `omp critical` y no `atomic`: el test de cota y el incremento tienen que ser un solo paso indivisible.

El test **E01** verifica este modelo aislado, en 0-D, contra los valores publicados de ten Tusscher 2004 — APD90, pico, reposo, dV/dt máximo. Es el test más barato de la suite y sostiene todo lo demás: si la corriente iónica estuviera mal, ningún test de convergencia lo detectaría, porque todos convergerían limpiamente a la respuesta equivocada.

```
1973     void protectState
1974     (
1975         scalarField& state,
1976         const scalar,
1977         const label,
1978         const char*
1979     )
1980     {
1981         if (!numericalProtection_) return;
1982
1983         auto correct = [&](const label i, const scalar value)
1984         {
1985             if (state[i] != value)
1986             {
1987                 state[i] = value;
1988                 recordCorrection();
1989             }
1990         };
1991
1992         if (!std::isfinite(state[V]))
1993         {
```

```

1994         correct(V, -86.2);
1995     }
1996     if (clampVmForRates_)
1997     {
1998         correct(V, min(max(state[V], vmMinForRates_), vmMaxForRates_));
1999     }
2000     if (mag(state[V]) < vmZeroEps_)
2001     {
2002         correct(V, state[V] < 0.0 ? -vmZeroEps_ : vmZeroEps_);
2003     }
2004
2005     const label concentrationIDs[] = {K_i, Na_i, Ca_i, Ca_SR};
2006     for (const label id : concentrationIDs)
2007     {
2008         if (!std::isfinite(state[id]) || state[id] < concentrationFloor_)
2009         {
2010             correct(id, concentrationFloor_);
2011         }
2012     }
2013
2014     if (clampGates_)
2015     {
2016         const label gateIDs[] = {Xr1, Xr2, Xs, m, h, j, d, f, fCa, s, r, g};
2017         for (const label id : gateIDs)
2018         {
2019             if (!std::isfinite(state[id]))
2020             {
2021                 correct(id, 0.0);
2022             }
2023             else
2024             {
2025                 correct(id, min(max(state[id], scalar(0.0)), scalar(1.0)));
2026             }
2027         }
2028     }
2029 }

```

Listing 1: La protección numérica y su contador compartido.

```

2512 void calculateCurrent
2513 (
2514     const scalar stepStartTime,
2515     const scalar,
2516     const scalarField& Vm,
2517     scalarField& Im,
2518     Field<Field<scalar>>& states
2519 )
2520 {
2521     const scalar tStartMs = 1000.0*stepStartTime;
2522     if (Im.size() != Vm.size())
2523     {
2524         FatalErrorInFunction
2525             << "Im.size() != Vm.size()" << abort(FatalError);
2526     }
2527
2528     // A mismatch here is a programming error, not something to repair
2529     // on the fly: re-initialising would discard the accumulated state
2530     // history. The state array must already be sized to the point set
2531     // it is integrated on (mesh.nCells() for cellCentredReconstruct,
2532     // the Iion Gauss points otherwise).
2533     if (states.size() != Vm.size())
2534     {
2535         FatalErrorInFunction
2536             << "states.size() = " << states.size()
2537             << " != Vm.size() = " << Vm.size() << nl
2538             << "The ionic state array must be sized to the point set "
2539             << "it is integrated on." << abort(FatalError);
2540     }
2541
2542     forAll(Vm, pointI)
2543     {
2544         scalarField& y = states[pointI];
2545         y[V] = 1000.0*Vm[pointI];
2546         protectState(y, tStartMs, pointI, "calculateCurrent");
2547         computeVariables(tStartMs, y, rates_[pointI], algebraic_[pointI]);

```

```

2548         Im[pointI] = algebraic_[pointI][Iion_cm];
2549     }
2550 }

```

Listing 2: La corriente iónica. Recortada.

3. Los integradores de EDO, y el único paralelismo por hilos

El solver trae sus propios steppers en vez de usar el marco ODESolver de OpenFOAM: `eulerStep`, `rk4Step` y `rkf45Trial` adaptativo. Un nombre desconocido es `FatalError`, no un fallback silencioso.

Euler y RK4 dan **un** paso del tamaño del paso exterior completo, así que su exactitud la controla `dt`; RKF45 subdivide por su cuenta y la controla su tolerancia. El test **E02** hace converger los tres al mismo potencial de acción, que es una afirmación mucho más fuerte que apretar uno solo contra sí mismo.

El paso adaptativo **persiste entre llamadas** en `stepMs_`, de modo que cada llamada hereda el h de la anterior. Eso hacía que la clave `initialODEStep` fuera inalcanzable — el constructor siembra `stepMs_` positivo, así que la rama que la leía sólo podía dispararse con `deltaT == 0`. El test **E03** lo midió (cero efecto sobre cuatro órdenes de magnitud) y la clave se removió.

`solveODE` contiene la **única** región OpenMP del archivo. Los hilos reparten los puntos de integración, y funciona porque cada punto resuelve su propia EDO sin hablar con ningún otro — por eso el resultado es bit-idéntico con cualquier cantidad de hilos, cosa que **E20** mide dando exactamente cero. **El ensamblado no está paralelizado con hilos.**

```

2138 void rkf45Trial
2139 (
2140     const scalar tMs,
2141     const scalar hMs,
2142     const scalarField& y,
2143     scalarField& yFifth,
2144     scalarField& rates,
2145     scalarField& algebraic,
2146     const label pointI,
2147     scalar& err
2148 )
2149 {
2150     scalarField k1(NUM_STATES, 0.0), k2(NUM_STATES, 0.0), k3(NUM_STATES, 0.0);
2151     scalarField k4(NUM_STATES, 0.0), k5(NUM_STATES, 0.0), k6(NUM_STATES, 0.0);
2152     scalarField yt(NUM_STATES, 0.0), yFourth(NUM_STATES, 0.0);
2153
2154     computeRates(tMs, y, k1, algebraic, pointI, "RKf45_k1");
2155     forAll(y, i) yt[i] = y[i] + hMs*(1.0/5.0)*k1[i];
2156     computeRates(tMs + hMs/5.0, yt, k2, algebraic, pointI, "RKf45_k2");
2157     forAll(y, i) yt[i] = y[i] + hMs*((3.0/40.0)*k1[i] + (9.0/40.0)*k2[i]);
2158     computeRates(tMs + 3.0*hMs/10.0, yt, k3, algebraic, pointI, "RKf45_k3");
2159     forAll(y, i) yt[i] = y[i] + hMs*((3.0/10.0)*k1[i] - (9.0/10.0)*k2[i] + (6.0/5.0)*k3[i]);
2160     computeRates(tMs + 3.0*hMs/5.0, yt, k4, algebraic, pointI, "RKf45_k4");
2161     forAll(y, i) yt[i] = y[i] + hMs*((-11.0/54.0)*k1[i] + (5.0/2.0)*k2[i] - (70.0/27.0)*k3[i] +
(35.0/27.0)*k4[i]);
2162     computeRates(tMs + hMs, yt, k5, algebraic, pointI, "RKf45_k5");
2163     forAll(y, i) yt[i] = y[i] + hMs*((1631.0/55296.0)*k1[i] + (175.0/512.0)*k2[i] + (575.0/13824.0)*k3[
i] + (44275.0/110592.0)*k4[i] + (253.0/4096.0)*k5[i]);
2164     computeRates(tMs + 7.0*hMs/8.0, yt, k6, algebraic, pointI, "RKf45_k6");
2165
2166     err = 0.0;
2167     forAll(y, i)
2168     {
2169         yFifth[i] = y[i] + hMs*((37.0/378.0)*k1[i] + (250.0/621.0)*k3[i] + (125.0/594.0)*k4[i] +
(512.0/1771.0)*k6[i]);
2170         yFourth[i] = y[i] + hMs*((2825.0/27648.0)*k1[i] + (18575.0/48384.0)*k3[i] + (13525.0/55296.0)*
k4[i] + (277.0/14336.0)*k5[i] + 0.25*k6[i]);
2171         const scalar scale = odeAbsTol_ + odeRelTol_*max(mag(yFifth[i]), mag(yFourth[i]));
2172         err = max(err, mag(yFifth[i] - yFourth[i])/max(scale, SMALL));
2173     }
2174     rates = k6;

```

2175 | }

Listing 3: El paso adaptativo. Recortado.

```
2552 void solveODE
2553 (
2554     const scalar stepStartTime,
2555     const scalar deltaT,
2556     const scalarField& Vm,
2557     scalarField& Im,
2558     Field<Field<scalar>>& states,
2559     const labelUList& scratchIDs = emptyScratchIDs()
2560 )
2561 {
2562     const scalar tStartMs = 1000.0*stepStartTime;
2563     const scalar dtMs = 1000.0*deltaT;
2564     if (Im.size() != Vm.size())
2565     {
2566         FatalErrorInFunction
2567             << "Im.size() != Vm.size()" << abort(FatalError);
2568     }
2569
2570     // Added for cardiacFoam: the caller may pass an explicit scratch
2571     // slot for each point. This is what keeps the front-aware hybrid
2572     // correct. That path calls solveODE twice per iteration, once with
2573     // a compacted array of nF front Gauss points and once with a
2574     // compacted array of nS smooth cells, both indexed from 0. Without
2575     // a slot map both passes write algebraic_/rates_/stepMs_[0..n),
2576     // so the second silently overwrites the first's scratch even
2577     // though the two arrays describe different physical points.
2578     if (!scratchIDs.empty())
2579     {
2580         if (scratchIDs.size() != Vm.size())
2581         {
2582             FatalErrorInFunction
2583                 << "scratchIDs.size() = " << scratchIDs.size()
2584                 << " != Vm.size() = " << Vm.size()
2585                 << abort(FatalError);
2586         }
2587
2588         forAll(scratchIDs, i)
2589         {
2590             if (scratchIDs[i] < 0 || scratchIDs[i] >= rates_.size())
2591             {
2592                 FatalErrorInFunction
2593                     << "scratchIDs[" << i << "] = " << scratchIDs[i]
2594                     << " out of range [0, " << rates_.size() << "]"
2595                     << abort(FatalError);
2596             }
2597         }
2598     }
2599
2600     // A mismatch here is a programming error, not something to repair
2601     // on the fly: re-initialising would discard the accumulated state
2602     // history. The state array must already be sized to the point set
2603     // it is integrated on (mesh.nCells() for cellCentredReconstruct,
2604     // the Iion Gauss points otherwise).
2605     if (states.size() != Vm.size())
2606     {
2607         FatalErrorInFunction
2608             << "states.size() = " << states.size()
2609             << " != Vm.size() = " << Vm.size() << nl
2610             << "The ionic state array must be sized to the point set "
2611             << "it is integrated on." << abort(FatalError);
2612     }
```

Listing 4: El despacho de stepper y la región OpenMP. Recortado. (recortado; sigue en el fuente.)

4. El residuo de estados: una colectiva que colgaba

`maxStateRelativeL2Difference` da el cambio relativo máximo de los estados entre dos iteraciones no lineales, y es parte del criterio de parada.

Es una función **colectiva**: reduce dos veces por estado. Y ahí había un cuelgue. La versión anterior devolvía temprano cuando el arreglo de estados de un rank venía vacío — exactamente lo que le pasa a un rank cuya partición no tiene celdas — así que ese rank salía sin participar de las reducciones mientras los demás quedaban bloqueados esperando. **No es un número mal calculado: es que la corrida no termina.**

Está corregido de dos formas, porque cualquiera de las dos sola dejaba el cuelgue vivo: el rank vacío ahora cae hasta los lazos de acumulación (que iteran cero veces) y llega igual a los dos **reduce**, y el conteo de vueltas del lazo se reduce con **minOp**, porque salía de **current[0].size()**, que es rank-local y ni siquiera se puede leer en un rank vacío.

El test **E19** lo mide sobre una malla de dos celdas: antes moría en el timeout de 300 s a np=4 y np=8, ahora corre en 1 s, igual que el control serial.

```

3097     scalar maxStateRelativeL2Difference
3098     (
3099         const Field<Field<scalar>>& current,
3100         const Field<Field<scalar>>& previous,
3101         scalarField& stateResiduals
3102     )
3103     {
3104         stateResiduals = 0.0;
3105
3106         // Modified for cardiacFoam: this function is COLLECTIVE - it reduces
3107         // twice per state below - so every rank has to reach every reduction,
3108         // and reach it the same number of times.
3109         //
3110         // What was here before returned 0.0 early when a rank's state array came
3111         // up empty, which is exactly what happens to a rank whose partition has
3112         // no cells. That rank skipped both reductions while the others blocked
3113         // in them: not a wrong number, a HANG. Reproduced on a 2-cell mesh with
3114         // 'simple (4 1 1)' and with 'scotch' at np=8 - both killed at the
3115         // timeout, while the same case at np=1 finishes in one second (test
3116         // E19).
3117         //
3118         // So the empty case is now handled by letting the accumulation loops
3119         // below contribute nothing: 'forAll(current, ...)' over an empty field
3120         // iterates zero times, num and den stay 0, and the rank still takes part
3121         // in every reduce.
3122         //
3123         // The trip count needs the same treatment. 'current[0].size()' is
3124         // rank-local and cannot even be read on an empty rank, so it is reduced:
3125         // otherwise two ranks could run this loop a different number of times
3126         // and desynchronise the reductions for the same reason.
3127         label nStates = stateResiduals.size();
3128         if (!current.empty())
3129         {
3130             nStates = min(current[0].size(), stateResiduals.size());
3131         }
3132         reduce(nStates, minOp<label>());
3133
3134         scalar maxResidual = 0.0;
3135
3136         for (label stateI = 0; stateI < nStates; ++stateI)
3137         {
3138             scalar num = 0.0;
3139             scalar den = 0.0;
3140
3141             forAll(current, pointI)
3142             {
3143                 if (stateI >= current[pointI].size() || stateI >= previous[pointI].size())
3144                 {
3145                     continue;
3146                 }
3147
3148                 const scalar c = current[pointI][stateI];
3149                 const scalar diff = c - previous[pointI][stateI];
3150                 num += diff*diff;
3151                 den += c*c;
3152             }
3153
3154             reduce(num, sumOp<scalar>());

```

```
3155 |         reduce(den, sumOp<scalar>());
```

Listing 5: La colectiva, con las dos correcciones. (*recortado; sigue en el fuente.*)

5. Bloques de construcción del ensamblado

Antes de las matrices de rigidez vienen las piezas compartidas.

`addTripletIfNeeded` es el punto por el que pasa toda entrada de la matriz, y descarta las menores que `SMALL`.

`addCellGradientDotCoeffs` agrega la fila de gradiente de una celda proyectada sobre un vector, usando los stencils MLS. Es donde vive el problema abierto más grande: sobre celdas pegadas a un corte de procesador ese stencil pierde un miembro, y como el ajuste se rehace sobre el conjunto reducido, **refita toda la fila**. Vive en `solids4foam`, no acá.

`exchangeCoupledFaceRows` intercambia con el rank vecino las filas que el término de estabilización necesita del otro lado del corte. Es **colectiva** y está deliberadamente *fuera* del lazo de chunks: adentro se trabaría en cuanto dos ranks tuvieran distinta cantidad de chunks.

```
3457 |     void addTripletIfNeeded
3458 |     (
3459 |         std::vector<Triplet>& triplets,
3460 |         const label row,
3461 |         const label col,
3462 |         const scalar value
3463 |     )
3464 |     {
3465 |         if (mag(value) > SMALL)
3466 |         {
3467 |             triplets.emplace_back(row, col, value);
3468 |         }
3469 |     }
```

Listing 6: Por acá pasa toda entrada.

```
3986 |     void exchangeCoupledFaceRows
3987 |     (
3988 |         const fvMesh& mesh,
3989 |         const std::function
3990 |         <
3991 |             void(const label patchI, const label faceI, labelList&, scalarField&)
3992 |         >& buildRow,
3993 |         List<labelList>& nbrCols,
3994 |         List<scalarField>& nbrCoeffs
3995 |     )
3996 |     {
3997 |         nbrCols.setSize(mesh.nBoundaryFaces());
3998 |         nbrCoeffs.setSize(mesh.nBoundaryFaces());
3999 |
4000 |         if (!Pstream::parRun())
4001 |         {
4002 |             return;
4003 |         }
4004 |
4005 |         PstreamBuffers pBufs(Pstream::commsTypes::nonBlocking);
4006 |
4007 |         forAll(mesh.boundary(), patchI)
4008 |         {
4009 |             const fvPatch& patch = mesh.boundary()[patchI];
4010 |             const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
4011 |
4012 |             if (!ppPtr)
4013 |             {
4014 |                 continue;
4015 |             }
4016 |
4017 |             List<labelList> sendCols(patch.size());
4018 |             List<scalarField> sendCoeffs(patch.size());
```

```

4019     forAll(patch, faceI)
4020     {
4021         buildRow(patchI, faceI, sendCols[faceI], sendCoeffs[faceI]);
4022     }
4023
4024
4025     UOPstream toNbr(ppPtr->neighbProcNo(), pBufs);
4026     toNbr << sendCols << sendCoeffs;
4027 }
4028
4029 pBufs.finishedSends();
4030
4031 forAll(mesh.boundary(), patchI)
4032 {
4033     const fvPatch& patch = mesh.boundary()[patchI];
4034     const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
4035
4036     if (!ppPtr)
4037     {
4038         continue;
4039     }
4040
4041     List<labelList> recvCols;
4042     List<scalarField> recvCoeffs;
4043
4044     UIPstream fromNbr(ppPtr->neighbProcNo(), pBufs);
4045     fromNbr >> recvCols >> recvCoeffs;

```

Listing 7: El intercambio a través del corte. Recortado. (*recortado; sigue en el fuente.*)

6. Las dos matrices de rigidez, y el defecto de la conductividad

Igual que en el MMS hay dos ensamblados y la tríada elige cuál corre: el ortogonal de dos puntos y el de alto orden sobre el stencil LRE.

Acá estuvo el defecto más grande que encontró la verificación. El camino de alto orden usaba `conductivity[own]` en las dos filas de una cara interna, en vez del promedio $\frac{1}{2}(D_{own} + D_{nei})$ que ya usaban el camino de bajo orden y el término de estabilización.

Con D uniforme los dos tensores coinciden y la diferencia se cancela idénticamente, así que fue invisible durante toda la vida del código: todas las corridas usaban el tensor constante del benchmark. Con D **no uniforme** — o sea, en cuanto entren fibras — hacía que K cambiara un **21 %** al descomponer la malla, porque al cortar, cada rank pasa a ser dueño de su lado de la cara y ninguno coincide con lo que hizo la corrida serial.

El arreglo son tres cambios, no uno: el promedio en la cara interna, el promedio **también en la cara de procesador** (con el tensor remoto), y sacar el intercambio de conductividad de la condición de estabilización, porque ahora hace falta con cualquier alpha. Corregir sólo el primero arreglaba el operador serial y dejaba el paralelo roto. **M15** lo mide: de 2.108e-01 a 4.736e-15.

```

4559 void assembleHighOrderStiffnessMatrix
4560 (
4561     const fvMesh& mesh,
4562     const volTensorField& conductivity,
4563     const highOrderInterp& LREInterp,
4564     const scalar stabilisationAlpha,
4565     const label tripletsPerFaceReserve,
4566     const label faceChunk,
4567     SpMat& K
4568 )
4569 {
4570     const bool twoD = mesh.nGeometricD() == 2;
4571     const vectorField& C = mesh.C();
4572     const surfaceVectorField& Cf = mesh.Cf();
4573     const CompactListList<point>& faceQP = LREInterp.faceQuadPoints();
4574     const CompactListList<scalar>& faceQW = LREInterp.faceQuadWeightPhysical();
4575     const CompactListList<label>& faceStencils =

```



```

4576     LREInterp.globalFaceStencils();
4577     const List<CompactListList<vector>>& faceGradCoeffs =
4578         LREInterp.QRGradFaceGPCoeffs();
4579
4580     const scalarField& V = mesh.V();
4581     const labelUList& owner = mesh.owner();
4582     const labelUList& neighbour = mesh.neighbour();
4583
4584     const label nCells = mesh.nCells();
4585     const label nInternalFaces = neighbour.size();
4586
4587     std::vector<Triplet> triplets;
4588     const label reserveFaces = min(max(mesh.nFaces(), label(1)), faceChunk);
4589     triplets.reserve(reserveFaces*tripletsPerFaceReserve);
4590
4591     SpMat Klocal(nCells, gNCols());
4592     bool kInitialised = false;
4593
4594     auto flushChunk =
4595     [&]()
4596     {
4597         if (triplets.empty())
4598         {
4599             return;
4600         }
4601
4602         Klocal.setZero();
4603         Klocal.setFromTriplets(triplets.begin(), triplets.end());
4604         Klocal.makeCompressed();
4605
4606         if (!kInitialised)
4607         {
4608             K.swap(Klocal);
4609             Klocal.resize(nCells, gNCols());
4610             kInitialised = true;
4611         }
4612         else
4613         {
4614             K += Klocal;
4615         }
4616
4617         triplets.clear();
4618     };

```

Listing 8: El ensamblado de alto orden. Recortado. *(recortado; sigue en el fuente.)*

7. Modo híbrido: EDO donde el frente lo pide

Resolver la EDO en cada punto de Gauss es lo más directo y lo más caro; resolverla en el centro de celda y reconstruir a los puntos es mucho más barato pero falla donde el frente es abrupto, porque los estados del TNNP son rígidos.

`flagFrontCells` clasifica las celdas del frente por el rango de V_m entre sus puntos de Gauss, y `dilateFrontCells` agranda esa banda unas capas. El modo híbrido resuelve la EDO en los puntos de Gauss ahí y en el centro de celda en el resto.

El detalle de implementación que importa: la clasificación se calcula **una vez por paso** y se congela para todo el solve no lineal. Recalcularla por iteración hace que el interruptor frente/suave oscile entre iteraciones y convierte el punto fijo de Picard en un ciclo límite que nunca converge.

```

5603     void flagFrontCells
5604     (
5605         const fvMesh& mesh,
5606         const scalarField& VmIntegrationPoints,
5607         const highOrderInterp& LREInterp_Ion,
5608         const volScalarField& externalStimulusCurrent,
5609         const scalar vmRangeThreshold,
5610         boolList& frontCell
5611     )

```

```

5612 {
5613     const CompactListList<point>& cellQP = LREInterp_Iion.cellQuadPoints();
5614     label ip = 0;
5615     forAll(mesh.cells(), cellI)
5616     {
5617         scalar vMin = GREAT, vMax = -GREAT;
5618         forAll(cellQP[cellI], qI)
5619         {
5620             const scalar v = VmIntegrationPoints[ip];
5621             vMin = min(vMin, v);
5622             vMax = max(vMax, v);
5623             ++ip;
5624         }
5625         frontCell[cellI] =
5626             (vMax - vMin > vmRangeThreshold)
5627             || (mag(externalStimulusCurrent[cellI]) > SMALL);
5628     }
5629 }

```

Listing 9: La clasificación del frente. Recortada.

```

5640 void dilateFrontCells
5641 (
5642     const fvMesh& mesh,
5643     const label nLayers,
5644     boolList& frontCell
5645 )
5646 {
5647     if (nLayers <= 0)
5648     {
5649         return;
5650     }
5651
5652     const labelListList& cellCells = mesh.cellCells();
5653
5654     // Added for cardiacFoam: marker field used to carry the front flag
5655     // across processor boundaries. mesh.cellCells() stops at a processor
5656     // face, so without this the front band simply failed to grow across a
5657     // partition. Unlike the stabilisation gap this is an O(1) error, not a
5658     // shrinking surface effect: a cell classified front integrates a
5659     // different set of ODEs from one classified smooth.
5660     //
5661     // Constructing the field with zeroGradient does NOT put zeroGradient on
5662     // the processor patches: fvPatchField::New overrides the requested type
5663     // with the patch's own constraint type, so processor patches receive a
5664     // real processorFvPatchScalarField and correctBoundaryConditions()
5665     // performs the halo exchange. One exchange per dilation layer.
5666     autoPtr<volScalarField> markerPtr;
5667     if (Pstream::parRun())
5668     {
5669         markerPtr.reset
5670         (
5671             new volScalarField
5672             (
5673                 IOobject
5674                 (
5675                     "frontCellMarker",
5676                     mesh.time().timeName(),
5677                     mesh,
5678                     IOobject::NO_READ,
5679                     IOobject::NO_WRITE
5680                 ),
5681                 mesh,
5682                 dimensionedScalar("zero", dimless, 0.0),
5683                 zeroGradientFvPatchScalarField::typeName
5684             )
5685         );
5686     }
5687
5688     for (label layer = 0; layer < nLayers; ++layer)
5689     {
5690         boolList grown(frontCell);
5691         forAll(frontCell, cellI)
5692         {
5693             if (!frontCell[cellI])

```

```

5694     {
5695         continue;
5696     }
5697     const labelList& nbr = cellCells[cellI];
5698     forAll(nbr, j)
5699     {

```

Listing 10: La dilatación de la banda. Recortada. (*recortado; sigue en el fuente.*)

8. Tiempos de activación y muestreo del benchmark

Lo que el benchmark reporta son **tiempos de activación**, no campos.

`updateActivationTimes` los interpola *dentro* del paso — $t_n + w \Delta t$ con w el cruce lineal del umbral — así que no quedan cuantizados a múltiplos de Δt , lo cual importa para cualquier estudio de refinamiento.

`writeActivationSamples` escribe el valor en el punto P8 y un perfil a lo largo de la diagonal. El muestreo es en puntos **físicos fijos**, independientes de la malla, y eso es lo que permite restar dos perfiles índice a índice sin interpolar entre discretizaciones — la base de E11 y de los estudios de convergencia.

`nearestCellToPoint` merece atención: devuelve el índice en **exactamente un** rank y -1 en todos los demás, resuelto con una reducción para que la elección sea determinista. El consumidor está guardado contra ese -1 ; el *banner* que imprime el punto de parada no lo estaba, y evaluaba `mesh.C() [-1]` en todos los ranks perdedores. En build optimizado eso es lectura fuera de rango, o sea comportamiento indefinido, y se manifestaba como **SIGFPE** de forma reproducible pero aparentemente arbitraria: a $dx=0.125$ mm fallaba con 4 ranks y no con 1, 2 ni 8.

```

5766 void updateActivationTimes
5767 (
5768     const scalarField& VmOld,
5769     const volScalarField& Vm,
5770     volScalarField& activationTime,
5771     boolList& calculateActivationTime,
5772     const scalar t0,
5773     const scalar dt,
5774     const scalar activationThreshold
5775 )
5776 {
5777     const scalarField& VmI = Vm.primitiveField();
5778     scalarField& activationTimeI = activationTime.primitiveFieldRef();
5779
5780     forAll(activationTimeI, cellI)
5781     {
5782         if (!calculateActivationTime[cellI])
5783         {
5784             continue;
5785         }
5786
5787         if (VmOld[cellI] < activationThreshold && VmI[cellI] >= activationThreshold)
5788         {
5789             const scalar denom = VmI[cellI] - VmOld[cellI];
5790             scalar w = 0.0;
5791             if (mag(denom) > VSMALL)
5792             {
5793                 w = (activationThreshold - VmOld[cellI])/denom;
5794             }
5795
5796             w = min(max(w, scalar(0.0)), scalar(1.0));
5797             activationTimeI[cellI] = t0 + w*dt;
5798             calculateActivationTime[cellI] = false;
5799         }
5800     }
5801
5802     activationTime.correctBoundaryConditions();

```

```
5803 }
```

Listing 11: La interpolación dentro del paso. Recortada.

```
5807 label nearestCellToPoint
5808 (
5809     const fvMesh& mesh,
5810     const point& samplePoint
5811 )
5812 {
5813     const vectorField& C = mesh.C();
5814     label bestCellI = -1;
5815     scalar bestD2 = GREAT;
5816
5817     forAll(C, cellI)
5818     {
5819         const scalar d2 = magSqr(C[cellI] - samplePoint);
5820         if (d2 < bestD2)
5821         {
5822             bestD2 = d2;
5823             bestCellI = cellI;
5824         }
5825     }
5826
5827     // Modified for cardiacFoam: the search above is over LOCAL cells only,
5828     // so every rank used to believe it owned the sample point. Downstream
5829     // that meant each rank triggered the stop condition on ITS OWN nearest
5830     // cell, clipped effectiveEndTime differently, and left the time loop at
5831     // a different step - some ranks dropping out of the collective solve
5832     // while the others blocked in it. A hang, preceded by corrupted physics,
5833     // because effectiveEndTime also sets the final step's dt.
5834     //
5835     // Exactly one rank now returns a cell; the others return -1. Ties are
5836     // broken by lowest rank, so the choice is deterministic and independent
5837     // of the decomposition. reduce() is preferred over Pstream::broadcast,
5838     // which broadcasts from masterNo() rather than from an arbitrary root.
5839     if (Pstream::parRun())
5840     {
5841         scalar globalBestD2 = bestD2;
5842         reduce(globalBestD2, minOp<scalar>());
5843
5844         label winner =
5845             (bestCellI >= 0 && bestD2 <= globalBestD2)
5846             ? Pstream::myProcNo()
5847             : Pstream::nProcs();
5848         reduce(winner, minOp<label>());
5849
5850         if (winner >= Pstream::nProcs())
5851         {
5852             FatalErrorInFunction
5853                 << "Could not find a cell for point " << samplePoint
5854                 << " on any rank" << exit(FatalError);
5855         }
5856
5857         return (winner == Pstream::myProcNo()) ? bestCellI : -1;
5858     }
5859
5860     if (bestCellI < 0)
5861     {
5862         FatalErrorInFunction
5863             << "Could not find a cell for point " << samplePoint
5864             << exit(FatalError);
5865     }
```

Listing 12: Exactamente un rank gana; el resto devuelve -1. (recortado; sigue en el fuente.)

9. main(): montaje y lazo temporal

`main()` lee malla y diccionarios, construye los operadores LRE, ensambla M y K **una sola vez** — la no linealidad está en el término iónico, no en la difusión — y entra al lazo temporal.

Cada paso resuelve un sistema no lineal con Picard, JFNK o diagonalion, y **E10** verifica que los tres lleguen al mismo lugar. Un nombre de método inválido es **FatalError** y no un fallback silencioso a Picard, cosa que **E18** comprueba.

Al agotar las iteraciones sin converger, la rama está **una sola vez**, después del despacho de método, así que los tres comparten la semántica: por defecto se descarta el paso y se vuelve a los valores del inicio, y con **nonlinearAcceptUnconverged** se acepta el iterado. **E24** lo verifica en las dos configuraciones. El solver MMS tiene el corte adentro del despacho y por eso allá Picard acepta mientras los otros dos descartan — dos semánticas en la misma matriz de métodos, y sigue abierto.

```

2767     scalar computeStableDeltaT
2768     (
2769         const volTensorField& conductivity,
2770         const scalar chiVal,
2771         const scalar CmVal,
2772         const scalar CFL,
2773         const scalar dx,
2774         const label dim
2775     )
2776     {
2777         const scalar Dxx = max(conductivity.component(tensor::XX)).value();
2778         const scalar Dyy = max(conductivity.component(tensor::YY)).value();
2779         const scalar Dzz = max(conductivity.component(tensor::ZZ)).value();
2780         const scalar Dmax = max(Dxx, max(Dyy, Dzz));
2781         const scalar DefMax = Dmax/(chiVal*CmVal);
2782
2783         scalar dtStable = GREAT;
2784         if (dim > 0 && DefMax > SMALL)
2785         {
2786             dtStable = CFL*sqr(dx)/(scalar(dim)*DefMax);
2787         }
2788
2789         reduce(dtStable, minOp<scalar>());
2790         return dtStable;
2791     }

```

Listing 13: El paso estable de referencia.

```

2823 void applyStimulus
2824 (
2825     const scalar t0,
2826     volScalarField& externalStimulusCurrent,
2827     const List<labelList>& stimulusCellIDsList,
2828     const List<scalar>& stimulusStartTimes,
2829     const scalar stimulusIntensity,
2830     const scalar stimulusDuration
2831 )
2832 {
2833     scalarField& extI = externalStimulusCurrent.primitiveFieldRef();
2834     extI = 0.0;
2835
2836     forAll(stimulusCellIDsList, bI)
2837     {
2838         const scalar tStart = stimulusStartTimes[bI];
2839         if (t0 < tStart || t0 > (tStart + stimulusDuration))
2840         {
2841             continue;
2842         }
2843
2844         const labelList& stimulusCellIDs = stimulusCellIDsList[bI];
2845         forAll(stimulusCellIDs, cI)
2846         {
2847             extI[stimulusCellIDs[cI]] = stimulusIntensity;
2848         }
2849     }
2850
2851     externalStimulusCurrent.correctBoundaryConditions();
2852 }

```

Listing 14: El estímulo a nivel tejido. Recortado.

10. Índice de funciones

Todas las funciones del espacio de nombres anónimo, en orden de aparición. Las secciones anteriores recorren las principales; ésta es el mapa completo para ubicarse en el archivo.

línea	función	líneas
94	gCol	4
100	gNCols	4
148	checkPetscError	10
170	buildDistributedMat	85
287	clear	15
303	reset	19
370	gSquaredNorm	6
379	gNorm	4
387	gDot	6
396	lowerWord	12
411	usesPetscBackend	5
422	petscKspTypeName	25
452	petscPcTypeName	43
506	petscParallelPcTypeName	13
522	isGmresType	4
581	copyEigVecToPetscVec	12
595	copyPetscVecToEigVec	16
615	setPetscOptionsPrefix	11
808	updateValues	62
871	solve	42
926	petscShellMatMult	17
955	petscShellPCApply	17
1037	initialise	112
1499	embeddedComputeStimulus	43
1543	computeIstim	14
1911	tissueFlag	14
1926	stateIndex	8
1935	algebraicIndex	8
1944	loadStimulusProtocol	10
1960	recordCorrection	12
1973	protectState	57
2031	computeRates	40
2072	computeVariables	19
2092	eulerStep	16
2109	rk4Step	28
2138	rkf45Trial	38
2177	advancePoint	133
2316	emptyScratchIDs	5
2469	initialiseStates	32
2502	updateStatesOld	4
2507	resetStatesToStatesOld	4
2512	calculateCurrent	39
2552	solveODE	108
2661	exportStates	31
2680	if	4
2693	exportStatesIntegrationPoints	38

línea	función	líneas
2738	characteristicDx	24
2767	computeStableDeltaT	25
2795	currentPeakRSSMB	14
2813	logMemoryCheckpoint	5
2823	applyStimulus	30
2863	computeHighOrderLaplacian	80
2946	quadraticForm	10
2964	reconstructFromTaylor	24
2992	thetaFromScheme	18
3013	normalizedMassMatrixType	18
3035	fieldToEigVec	12
3051	eigVecToField	9
3063	relativeL2Difference	21
3087	relativeL2Norm	5
3097	maxStateRelativeL2Difference	66
3176	diagonalBlock	23
3203	addDiagonalToMatrix	18
3235	solveGMRES	142
3390	postProcessingDir	11
3416	reportOperatorFingerprint	35
3457	addTripletIfNeeded	13
3473	assembleDiagonalMassMatrix	31
3517	assembleConsistentMassMatrixHO	141
3663	orthogonalDiffusionCoeff	14
3680	normalDiffusivity	4
3690	stabilisationFaceCoeff	17
3711	addCellGradientDotCoeffs	58
3786	cellTaylorEntryCoeff	32
3827	addCellTaylorExtrapolationCoeffs	57
3888	buildCellTaylorExtrapolationRow	29
3921	buildCellGradientDotRow	26
3951	addRemoteRowCoeffs	19
3986	exchangeCoupledFaceRows	78
4077	addRhieChowInternalJumpCoeffs	34
4118	addRhieChowCoupledJumpCoeffs	38
4161	addRhieChowBoundaryJumpCoeffs	21
4194	assembleStandardOrthogonalStiffnessMatrix	345
4559	assembleHighOrderStiffnessMatrix	478
5046	solveSparseSystemEigen	94
5149	solveSparseSystem	91
5250	reconstructVmAtIionIntegrationPoints	80
5340	reconstructStatesAtIionIntegrationPoints	138
5492	clampPhysical	13
5509	averageIionIntegrationPointsToCells	34
5548	averageStatesIntegrationPointsToCells	47
5603	flagFrontCells	27
5640	dilateFrontCells	116
5766	updateActivationTimes	38
5807	nearestCellToPoint	62
5875	sampleIDW	146
6025	writeActivationSamples	63

línea	función	líneas
6090	<code>main</code>	2055